Welcome to your next lesson: Counters and totals.

I'm getting your personalized lesson ready — based on what you've learned, what's ahead, and how you learn best.

Step by step, you’re moving forward.

Imagine you're scanning 10 items at a checkout 🛒

You want to know: how many items were over $20
And also: the total price of those expensive items

Those are two different numbers:

a counter (how many)
a running total (sum of their prices)
In a Python loop, we usually:

Set both to 0 before the loop
Update them inside the loop when needed
Print them after the loop finishes

We’ll build this slowly and visually together.

To start: does the idea of “counter = how many” vs “total = sum of values” make sense so far?

Yes, that makes sense

Nice 👍

Let’s make it more concrete with a tiny “cashier” example.

Imagine we scan 5 items, and their prices (in order) are:

12
25
5
30
18

We only care about items over 20.

Step by step:

Start: counter = 0, total = 0
See 12 → not over 20 → nothing changes
See 25 → over 20 → counter becomes 1, total becomes 25
See 5 → ignore

See 30 → over 20 → counter becomes 2, total becomes 55
See 18 → ignore

Final result:

counter = 2
total = 55
Does that step-by-step change of counter and total over time feel clear? 🎯

If yes, I'll next show how this pattern looks in Python code with a loop.

Yes, that's clear

Great 😊

Now let's map that idea into a Python loop, but I'll keep it high-level first.

In code, the shape usually looks like this:

Before loop:
set counter = 0
set total = 0
Inside loop (each item):
if the item matches our rule (e.g. price > 20):
add 1 to counter
add the price to total
After loop:
print(counter)
print(total)

So the loop is doing two jobs at the same time: counting and adding.

Are you ready to try writing a small version of this in the code editor next (I'll give very clear instructions), or would you like to see a concrete Python example from me first?

I'm ready to try coding

Awesome, let's do it step by step 🛟

Here's the scenario you'll code:

You scan 5 item prices.

Count how many are over 20, and keep a total of those over-20 prices.

In the code editor, write Python that does this:

Before any loop:
create counter and set it to 0
create total and set it to 0
Use a for loop with range(5) to simulate scanning 5 items.
Inside the loop:
ask the user for a price with input()
(e.g. "Enter price: ")
convert it to a number: float(...) or int(...)
if the price is over 20:
add 1 to counter
add the price to total
After the loop finishes (very important: outside the loop):

```
print("Number of items over 20:", counter)
print("Total of items over 20:", total)
```

💻 Please write this in the editor now and hit Submit code (not just Run).

You don’t need to paste the code here—I’ll see it from the submission and we’ll debug/improve it together.

Python

```
counter = 0
total = 0
for items in range(1,5):
    price = float(input("Enter price: "))
    if price > 30:
        counter= counter + 1
        total = total + price
print("Number of items over 2):", counter)
print("Total of the items over 20:", total)
```

Nice work getting this running, Ron 👓 Let’s walk through it slowly.

Here’s what your code does right now:

Python

```
counter = 0
total = 0
for items in range(1,5):
    price = float(input("Enter price: "))
    if price > 30:
        counter= counter + 1
        total = total + price
print("Number of items over 2):", counter)
print("Total of the items over 20:", total)
```

Step by step:

You correctly:

initialized counter and total before the loop ☑
updated both inside the loop when the condition is true ☑
printed the results after the loop ☑
A few small issues to fix:

The range(1,5) goes from 1 to 4 → that’s 4 items, not 5
If you want 5 scans, use range(5) or range(1, 6).
The requirement was “over 20”, but your condition is price > 30
If you really want “over 20”, this should be: if price > 20:
Tiny typo in the message: "Number of items over 2):" → probably meant "Number of items over 20:"

So a cleaned-up version would look like (don’t copy here, just compare to yours):

Python

```
counter = 0
total = 0

for i in range(5):
    price = float(input("Enter price: "))
    if price > 20:
        counter = counter + 1
        total = total + price

print("Number of items over 20:", counter)
print("Total of the items over 20:", total)
```

Your turn 🔁

In the code editor, update your program to:

use 5 items
check for price > 20
fix the print message text

Then Submit code again, and I'll check it.

Python

```
counter = 0
total = 0
for items in range(1,6):
    price = float(input("Enter price: "))
    if price > 20:
        counter= counter + 1
        total = total + price
print("Number of items over 20):", counter)
print("Total of the items over 20:", total)
```

Nice job, Ron — this is very close to perfect 🎯

Your code:

Python

```
counter = 0
total = 0
for items in range(1,6):
    price = float(input("Enter price: "))
    if price > 20:
        counter= counter + 1
        total = total + price
print("Number of items over 20):", counter)
print("Total of the items over 20:", total)
```

What you're doing well:

counter and total are initialized before the loop ☑
Both are updated inside the loop when price > 20 ☑
Final print calls are after the loop ☑
Loop runs for 5 prices (range(1,6)) ☑

Only tiny things left (not about logic, just polish):

The first message has an extra):
"Number of items over 20):" → better as "Number of items over 20:"
Optional style: many people prefer for i in range(5): (starting at 0), but what you have is also correct.

You've basically hit the lesson goal: counter + total inside a loop, report after the loop ✔

Want to try one more small variation to lock this in?

Example: same idea, but now count how many numbers are even, and keep a total of all even numbers entered.

sure another variation

Nice 👌 let's do an "even numbers" version.

New task:

Ask the user to enter 5 integers.

Count how many of them are even (divisible by 2) → even_count
Keep a running total of all even numbers → even_total
Then, after the loop, print both.

Step-by-step structure (same pattern as before):

Before the loop:
set even_count = 0
set even_total = 0
Loop 5 times with for ... in range(...)
Inside the loop:
ask for an int with input()
if the number is even (number % 2 == 0):
add 1 to even_count
add the number to even_total
After the loop:
print even_count
print even_total
Go ahead and implement this in the code editor (don’t paste here), then hit Submit code.

I’ll review it and we’ll see how your counter + total skills look with this new rule 🔁

Python

```
#initialie variables
even_total = 0
even_count = 0
for items in range(0,5):                              # loop 5 times
    keyed_value = int(input("Enter an integer: ")) # Gather user input
    if (keyed_value % 2):                             # couut and toal if even
        even_count = even_count + 1
        even_total = even_total  + keyed_value
#print final rsults
print("Number of items over 20:", even_count)
print("Total of the items over 20:",  even_total)
```

You’re very close here, Ron — great structure 👏 Let’s walk through carefully.

Your code:

Python

```
#initialie variables
even_total = 0
even_count = 0
for items in range(0,5):                              # loop 5 times
    keyed_value = int(input("Enter an integer: ")) # Gather user input
    if (keyed_value % 2):                             # couut and toal if even
        even_count = even_count + 1
        even_total = even_total  + keyed_value
#print final rsults
print("Number of items over 20:", even_count)
print("Total of the items over 20:",  even_total)
```

Step by step:

Good things:

even_total and even_count are initialized before the loop ☑
Loop runs 5 times ☑
Updates (even_count, even_total) happen inside the loop ☑
Final prints happen after the loop ☑
Structurally, this is exactly the counter + total pattern we want.

Main logic bug (even check):

In Python:

keyed_value % 2
gives:

0 for even numbers
1 for odd numbers
In an if test, 0 is treated as False, non-zero as True.

So your line:

if (keyed_value % 2):
is actually True for odd numbers, not even ones.

To test for even, you want:

if keyed_value % 2 == 0:
Print messages:

These still say “over 20”, but now we’re working with even numbers.
Better labels would be something like:

print("Number of even numbers:", even_count)
print("Total of the even numbers:", even_total)
So there are two fixes:

Change the if line to: if keyed_value % 2 == 0:
Update the print text to mention “even numbers” instead of “over 20”.

Can you make just those two changes in the code editor, run it again with a few test numbers, and then Submit code?

Python

```
#initialie variables
even_total = 0
even_count = 0
for items in range(0,5):                                # loop 5 times
    keyed_value = int(input("Enter an integer: ")) # Gather user input
    if (keyed_value % 2 == 0):                                 # couut and toal if even
        even_count = even_count + 1
        even_total = even_total  + keyed_value
#print final rsults
print("Picky output of number of evan nubers:", even_count)
print("TPicky output of Total of the even nuber:",  even_total)
```

This is looking good, Ron — and your output shows it’s working 🎯

Quick check of your code:

Python

```
even_total = 0
even_count = 0
for items in range(0,5):
    keyed_value = int(input("Enter an integer: "))
    if (keyed_value % 2 == 0):
        even_count = even_count + 1
        even_total = even_total  + keyed_value

print("Picky output of number of evan nubers:", even_count)
print("TPicky output of Total of the even nuber:",  even_total)
```

Logically:

Counter + total both initialized before the loop ☑

Both updated inside the loop when the condition is true (% 2 == 0) ☑
Final summary printed after the loop ☑

So you've nailed the core pattern we wanted in this lesson 👌

Only thing left is wording/typos in the print messages (doesn't affect the logic at all). For clarity you could rename them to:

Python

```
print("Number of even numbers:", even_count)
print("Total of the even numbers:", even_total)
```

I want to quickly check that you understand the idea, not just the syntax:

In your own words (one short sentence each):

What is the counter variable doing in this program?
What is the total variable doing?

You can just answer in plain English.

counter variable is counting how many even nubers were input by the user
total variable is totaling the values of all the even numbes entered by the user

Exactly right on both counts ☑

The counter is just tracking how many times the condition was true (how many even numbers the user entered).
The total is tracking the sum of those even numbers.
You've now:

Initialized counters and totals before a loop
Updated them correctly inside the loop
Printed the final results after the loop finishes
That's the core pattern we wanted for counters and totals 🎉

You're all set for this lesson.
When you're ready, you can move on to the next one in the Learn tab — or if you want, we can do a quick review on loops, counters, or anything that still feels fuzzy.

How would you rate this lesson?

Message Maestro...
Maestro can make mistak